Advanced Programming

1. Evolution of Java GUI (From AWT → Swing → JavaFX)

a) AWT (Abstract Window Toolkit)

What it is:

- The *first* Java GUI toolkit (Java 1.0).
- Provides basic GUI components: `Button`, `TextField`, `Label`, etc.
- Uses *native OS components* (Windows button, Mac button, etc.) → called **heavyweight**.

Goal:

- "Write Once, Run Anywhere" (WORA): one Java GUI works across all systems.

Why it failed:

- Only included *common* features that exist on all OS → called the **"lowest common denominator"** rule.
- Couldn't include advanced UI elements (Tables, Trees, Tabs, Tooltips, etc.).
- **Looked different** on each OS (Windows style on Windows, Mac style on Mac).
- **Limited customization** (you couldn't easily change colors, styles).
- **Platform bugs** and behavior differences.

 Summary Note:

AWT = First GUI, Heavyweight, Native components, Limited features, Platform-dependent.

---

b) Swing

Introduced: Java 1.2 (part of JFC – Java Foundation Classes).
Goal: Fix AWT's problems.
How:

- Components are drawn by **Java itself**, not the OS → **lightweight**.
- Has more UI elements: tables, trees, tabbed panes, etc.
- Customizable "Look and Feel" (pluggable theme system).

Pros:
 Consistent across platforms

 Rich components
 Customizable appearance

Cons:
 Sometimes felt slow (CPU rendering, no GPU acceleration)
 Complex to use for advanced graphics

 Summary Note:

Swing = Lightweight, pure Java, rich features, consistent look, customizable but slower for modern graphics.

---

c) JavaFX

Introduced: Around 2008, integrated into JDK 7.
Goal: Replace Swing and support modern UIs (animations, media, touch, GPU).
Modern features:

- GPU-accelerated graphics (smooth animations).
- **Scene Graph** structure (UI as a tree of nodes).
- **FXML**: XML-like layout design.
- **CSS styling** like web pages.
- **Binding system** (auto updates UI when data changes).
- Supports 2D/3D, media, and web content.
- Cross-platform (still WORA).

 Summary Note:

JavaFX = Modern GUI toolkit, GPU-based, uses Scene Graph, supports CSS/FXML/Animations, ideal for modern apps.

---

2. JavaFX Architecture

Scene Graph Concept

Think of it like a **tree**:

```
Stage (Window)
 └── Scene (Container)
      └── Root Node (e.g., VBox)
           ├── Button
           ├── Label
           └── ImageView
```

Nodes: Everything you see (buttons, shapes, text).

- **Branch Nodes (Parent):** Can contain other nodes → e.g., `VBox`, `HBox`, `BorderPane`.
- **Leaf Nodes:** Cannot contain others → e.g., `Button`, `Label`, `ImageView`.

Rendering Pipeline

- Automatically redraws only what changes.
- Uses **GPU acceleration** → smoother animations than Swing.

 Summary Note:

Scene Graph = Tree of nodes.
Branch nodes = containers; Leaf nodes = UI elements.
Rendering uses GPU → fast and smooth.

---

3. JavaFX Application Lifecycle

Every JavaFX program extends `Application`.

Method	Purpose	Runs On
`main()`	Starts the program, calls `Application.launch()`	Main thread
`init()`	Optional setup (load files, database, etc.)	Main thread
`start(Stage primaryStage)`	Build and show the UI (main method)	JavaFX Application Thread
`stop()`	Optional cleanup when closing	JavaFX Application Thread

 Summary Note:

Lifecycle = `main → init → start → stop`.
`start()` is the heart of your JavaFX program.

---

4. Stage vs Scene

Think of a **play**:

 Stage = the theater window (main window).
 Scene = the stage content (what's inside the window).

- **Stage:**
  - Provided by OS (top-level window).
  - Has title, size, icon, resizable property.
- **Scene:**
  - Holds the visual hierarchy (Scene Graph).
  - Can switch Scenes on the same Stage.

 Summary Note:

Stage = Window.
Scene = Content inside it (UI).
A Stage has one Scene at a time.

---

5. “Hello JavaFX” Example

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class HelloJavaFX extends Application {
    @Override
    public void start(Stage primaryStage) {
        Label helloLabel = new Label("Hello, JavaFX!");
        StackPane root = new StackPane(helloLabel);
        Scene scene = new Scene(root, 300, 200);
        primaryStage.setTitle("My First JavaFX App");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }

    @Override
    public void init() {
        System.out.println("Application initializing...");
    }

    @Override
    public void stop() {
        System.out.println("Application shutting down...");
    }
}
```

Step-by-Step Explanation

Step	Description
1	`Label` → creates a simple text UI element (Leaf Node).
2	`StackPane` → a layout (Branch Node) that centers its child.
3	`Scene` → holds the layout (root node) and size.
4	`Stage` → the main window; attach the scene to it.
5	`show()` → makes window visible.
6	`main()` → launches JavaFX runtime.
7	`init()` and `stop()` → optional setup and cleanup methods.

Summary Note:

`Label` → text
`StackPane` → layout
`Scene` → holds layout
`Stage` → window
`show()` → display
Lifecycle: `init → start → stop`

---

6. How to Compile and Run

Compile

```
javac --module-path "path_to_javafx/lib" --add-modules javafx.controls
HelloJavaFX.java
```

Run

```
java --module-path "path_to_javafx/lib" --add-modules javafx.controls
HelloJavaFX
```

 Summary Note:

Always add `--module-path` and `--add-modules javafx.controls` when compiling/running.

Key Takeaways for Quick Revision

Concept	Keyword Summary
AWT	Heavyweight, native components, limited, inconsistent
Swing	Lightweight, pure Java, customizable, CPU-based
JavaFX	Modern, GPU-based, CSS, FXML, animations
Scene Graph	Tree of Nodes (Branch + Leaf)
Stage	Window
Scene	Content of window
Lifecycle	`main → init → start → stop`
Hello JavaFX	Label + StackPane + Scene + Stage

3.2. JavaFX Layout Components — Overview

In JavaFX, **layout panes** are containers that automatically organize (arrange + size) their children (controls, buttons, labels, etc.).

You don't have to position everything manually — the **layout manager** does it for you.

□ Understanding Layout Concepts

1. Managed vs Unmanaged

- **Managed nodes**: Their parent layout (like VBox, GridPane, etc.) automatically decides their position and size.
- **Unmanaged nodes**: You manually control their position with `setLayoutX()` and `setLayoutY()`.

Manual (unmanaged) layout is rarely used in complex apps — it doesn't resize well when the window changes.

Example:

```
Button btn = new Button("Click Me");
btn.setManaged(false);
btn.setLayoutX(100);
btn.setLayoutY(50);
```

---

2 Node Sizing: Preferred, Minimum, Maximum

Each JavaFX node (like Button, TextField, etc.) has **three main size hints**:

Property	Meaning
`setMinWidth()` / `setMinHeight()`	Smallest allowed size
`setPrefWidth()` / `setPrefHeight()`	Desired (natural) size
`setMaxWidth()` / `setMaxHeight()`	Largest allowed size

Layout panes use these hints to arrange nodes.
If you use a **manual layout** (like Pane), then `setPrefSize(w,h)` defines the visible size directly.

---

3. Padding vs Spacing

- **Padding** = Empty space *inside* the pane, around its edges.
- **Spacing** = Empty space *between* the pane's children.

Example:

```
VBox box = new VBox(10); // spacing = 10px
box.setPadding(new Insets(20)); // padding = 20px around content
```

The Core Layout Panes

Now, let's go through each layout pane with easy understanding and visuals.

---

1. VBox (Vertical Box)

Arranges children **vertically (top to bottom)**.

Use for: Forms, menus, vertical lists.

Key Properties:

- `spacing` — vertical gap between elements.

- `padding` — space around all children.
- `alignment` — how children align horizontally (`Pos.CENTER_LEFT`, etc.).
- `VBox.setVgrow(node, Priority.ALWAYS)` — makes a child grow vertically when space allows.

Example:

```
VBox root = new VBox(10);
root.setPadding(new Insets(20));
root.setAlignment(Pos.CENTER_LEFT);
root.getChildren().addAll(nameLabel, nameField, emailLabel, emailField,
submitButton);
```

 VBox automatically stacks everything vertically.

---

2. HBox (Horizontal Box)

 Arranges children **horizontally (left to right)**.

Use for: Toolbars, rows of buttons, status bars.

Key Properties:

- `spacing` — horizontal space between children.
- `alignment` — vertical alignment within the box.
- `HBox.setHgrow(node, Priority.ALWAYS)` — allows stretching horizontally.

Example:

```
HBox root = new HBox(15);
root.setPadding(new Insets(10));
root.setAlignment(Pos.CENTER);
root.getChildren().addAll(newButton, openButton, saveButton, exitButton);
```

 Buttons align in one horizontal line.

---

3. GridPane

 Arranges children in **rows and columns**, like a table.

Use for: Forms, dashboards, login/registration screens.

Key Properties:

- `hgap`, `vgap` — space between cells.
- `GridPane.setColumnIndex(node, index)` / `setRowIndex(node, index)` — set position.
- `setColumnSpan()` / `setRowSpan()` — make a node occupy multiple cells.
- `ColumnConstraints` — control column resizing and width behavior.

Example:

```
gridPane.add(nameLabel, 0, 1);
gridPane.add(nameField, 1, 1);
```

 Left column = labels, right column = input fields.
You can also use an **HBox inside GridPane** to align buttons horizontally.

---

4. BorderPane

 Divides the screen into **five regions**:
Top, **Bottom**, **Left**, **Right**, and **Center**.

Use for: Main app layout (menus, content area, sidebars).

Example:

```
root.setTop(menuBar);
root.setLeft(sidebar);
root.setCenter(mainContent);
root.setBottom(statusBar);
```

 Each region can hold only **one direct child** — if you want more, put a VBox/HBox inside.

 Think of it as a website structure:

```
Top → Header
Left → Navigation
Center → Main Content
Right → Sidebar
Bottom → Footer
```

---

5. StackPane

Stacks all children on top of each other (like layers).

Use for: Overlays, image + text layers, loading screens.

Example:

```
StackPane root = new StackPane();
root.getChildren().addAll(backgroundImageView, overlayText);
StackPane.setAlignment(overlayText, Pos.BOTTOM_CENTER);
```

 The last child you add is **on top**.

---

6. FlowPane

 Arranges items **in a flow**, wrapping automatically when space runs out.

Use for: Tag lists, galleries, toolbars that wrap.

Example:

```
FlowPane root = new FlowPane(Orientation.HORIZONTAL, 10, 10);
for (int i = 1; i <= 20; i++) {
    root.getChildren().add(new Button("Button " + i));
}
```

 As the window shrinks, items move to the next line automatically.

---

7. AnchorPane

 Lets you **anchor elements** to the top, bottom, left, or right edges.

Use for: Fixed positioning (buttons in corners, headers, etc.).

Example:

```
AnchorPane.setTopAnchor(btn, 10.0);
AnchorPane.setRightAnchor(btn, 10.0);
```

 If you anchor opposite sides (left & right), the element stretches.

Nesting Layouts (Combining Them)

You can **nest layouts inside one another** to build complex UIs.

Example:

- BorderPane as main layout.

- VBox in the left (navigation).
- GridPane in the center (form).
- HBox at the top (title bar).

 Nesting = Power + Flexibility.

Summary Table

Layout Pane	Direction	Common Use	Key Properties
VBox	Vertical	Forms, menus	spacing, alignment
HBox	Horizontal	Toolbars, buttons	spacing, alignment
GridPane	Table-like	Forms	hgap, vgap, constraints
BorderPane	5 regions	App structure	setTop, setCenter, etc.
StackPane	Overlapping layers	Overlays	alignment, margin
FlowPane	Flow wrapping	Galleries	orientation, hgap, vgap
AnchorPane	Fixed edges	Toolbars	setTopAnchor, setBottomAnchor

3.2. JavaFX Layout Components — Overview

In JavaFX, layout panes are containers that automatically organize (arrange + size) their children (controls, buttons, labels, etc.).

You don’t have to position everything manually — the layout manager does it for you.

 Understanding Layout Concepts

1 Managed vs Unmanaged

Managed nodes: Their parent layout (like VBox, GridPane, etc.) automatically decides their position and size.

Unmanaged nodes: You manually control their position with setLayoutX() and setLayoutY().

 Manual (unmanaged) layout is rarely used in complex apps — it doesn't resize well when the window changes.

Example:

```
Button btn = new Button("Click Me");
btn.setManaged(false);
btn.setLayoutX(100);
btn.setLayoutY(50);
```

2 Node Sizing: Preferred, Minimum, Maximum

Each JavaFX node (like Button, TextField, etc.) has three main size hints:

Property	Meaning
setMinWidth() / setMinHeight()	Smallest allowed size
setPrefWidth() / setPrefHeight()	Desired (natural) size
setMaxWidth() / setMaxHeight()	Largest allowed size

 Layout panes use these hints to arrange nodes.

If you use a manual layout (like Pane), then setPrefSize(w,h) defines the visible size directly.

3 Padding vs Spacing

Padding = Empty space inside the pane, around its edges.

Spacing = Empty space between the pane’s children.

Example:

```
VBox box = new VBox(10); // spacing = 10px
box.setPadding(new Insets(20)); // padding = 20px around content
```

The Core Layout Panes

Now, let’s go through each layout pane with easy understanding and visuals.

1. VBox (Vertical Box)

Arranges children vertically (top to bottom).

Use for: Forms, menus, vertical lists.

Key Properties:

spacing — vertical gap between elements.

padding — space around all children.

alignment — how children align horizontally (Pos.CENTER_LEFT, etc.).

VBox.setVgrow(node, Priority.ALWAYS) — makes a child grow vertically when space allows.

Example:

```
VBox root = new VBox(10);
root.setPadding(new Insets(20));
root.setAlignment(Pos.CENTER_LEFT);
root.getChildren().addAll(nameLabel, nameField, emailLabel, emailField, submitButton);
```

 VBox automatically stacks everything vertically.

 2. HBox (Horizontal Box)

 Arranges children horizontally (left to right).

Use for: Toolbars, rows of buttons, status bars.

Key Properties:

spacing — horizontal space between children.

alignment — vertical alignment within the box.

HBox.setHgrow(node, Priority.ALWAYS) — allows stretching horizontally.

Example:

```
HBox root = new HBox(15);
root.setPadding(new Insets(10));
root.setAlignment(Pos.CENTER);
root.getChildren().addAll(newButton, openButton, saveButton, exitButton);
```

 Buttons align in one horizontal line.

 3. GridPane

 Arranges children in rows and columns, like a table.

Use for: Forms, dashboards, login/registration screens.

Key Properties:

hgap, vgap — space between cells.

GridPane.setColumnIndex(node, index) / setRowIndex(node, index) — set position.

setColumnSpan() / setRowSpan() — make a node occupy multiple cells.

ColumnConstraints — control column resizing and width behavior.

Example:

```
gridPane.add(nameLabel, 0, 1);
gridPane.add(nameField, 1, 1);
```

 Left column = labels, right column = input fields.

You can also use an HBox inside GridPane to align buttons horizontally.

 4. BorderPane

 Divides the screen into five regions:

Top, Bottom, Left, Right, and Center.

Use for: Main app layout (menus, content area, sidebars).

Example:

```
root.setTop(menuBar);

root.setLeft(sidebar);

root.setCenter(mainContent);

root.setBottom(statusBar);
```

 Each region can hold only one direct child — if you want more, put a VBox/HBox inside.

 Think of it as a website structure:

Top → Header

Left → Navigation

Center → Main Content

Right → Sidebar

Bottom → Footer

 5. StackPane

 Stacks all children on top of each other (like layers).

Use for: Overlays, image + text layers, loading screens.

Example:

```
StackPane root = new StackPane();
```

```
root.getChildren().addAll(backgroundImageView, overlayText);
StackPane.setAlignment(overlayText, Pos.BOTTOM_CENTER);
```

 The last child you add is on top.

 6. FlowPane

 Arranges items in a flow, wrapping automatically when space runs out.

Use for: Tag lists, galleries, toolbars that wrap.

Example:

```
FlowPane root = new FlowPane(Orientation.HORIZONTAL, 10, 10);
for (int i = 1; i <= 20; i++) {
    root.getChildren().add(new Button("Button " + i));
}
```

 As the window shrinks, items move to the next line automatically.

 7. AnchorPane

 Lets you anchor elements to the top, bottom, left, or right edges.

Use for: Fixed positioning (buttons in corners, headers, etc.).

Example:

```
AnchorPane.setTopAnchor(btn, 10.0);
AnchorPane.setRightAnchor(btn, 10.0);
```

 If you anchor opposite sides (left & right), the element stretches.

 Nesting Layouts (Combining Them)

You can nest layouts inside one another to build complex UIs.

Example:

BorderPane as main layout.

VBox in the left (navigation).

GridPane in the center (form).

HBox at the top (title bar).

 Nesting = Power + Flexibility.

 Summary Table

Layout Pane	Direction	Common Use	Key Properties
VBox	Vertical	Forms, menus	spacing, alignment
HBox	Horizontal	Toolbars, buttons	spacing, alignment
GridPane	Table-like	Forms	hgap, vgap, constraints
BorderPane	5 regions	App structure	setTop, setCenter, etc.
StackPane	Overlapping layers	Overlays	alignment, margin
FlowPane	Flow wrapping	Galleries	orientation, hgap, vgap
AnchorPane	Fixed edges	Toolbars	setTopAnchor, setBottomAnchor

3.3.1 Event Handlers — Simplified Breakdown

 What Are Event Handlers?

In any GUI (Graphical User Interface) app, event handling is how your program reacts to what the user does — like clicks, key presses, or mouse moves.

When you click a button, type in a field, or move the mouse — that's an event.

Event handlers are the code that runs when that event happens.

 JavaFX Event Flow

In JavaFX, every visual element (Button, Pane, etc.) is a node in the Scene Graph.

When an event happens (say a mouse click), it travels through that scene graph in two phases:

 1. Capturing (Filtering) Phase — From Root → Target

The event starts at the root node and travels down to the node that caused it.

You can “filter” it before it reaches the target by using:

```
node.addEventFilter(EventType, handler);
```

A filter can even stop the event completely using:

```
event.consume();
```

 Use case example:

You want to detect any click in a window before it reaches specific buttons — maybe for validation or blocking behavior.

 2. Bubbling Phase — From Target → Root

After reaching the target node, the event travels back up toward the root.

You can handle it here using:

```
node.addEventHandler(EventType, handler);
```

Or the simpler version:

```
button.setOnAction(event -> { ... });
```

 Use case example:

A button click event — most handlers you write (like setOnAction) happen during this phase.

 Key Terms Simplified

Term	Meaning
Event	The object describing what happened (e.g., MouseEvent, ActionEvent).
Event Source	The node that generated the event (e.g., the Button clicked).
Event Target	Usually the same as the source — the node where it happened.
Event Handler	The code that runs when the event occurs.
Event Filter	Code that intercepts the event before it reaches the target (during capturing).
event.consume()	Stops the event from continuing (no other handlers will get it).

 3 Ways to Implement Event Handling

1 Anonymous Inner Class (Old Style)

Verbose but explicit.

```
btn.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
```

```
        System.out.println("Button clicked!");
    }
});
```

 Use if you need to access local variables and prefer full syntax.

2 Separate Named Class

Reusable and organized.

```
class MyButtonHandler implements EventHandler<ActionEvent> {
    public void handle(ActionEvent e) {
        System.out.println("Handled in MyButtonHandler");
    }
}
btn.setOnAction(new MyButtonHandler());
```

 Use this when the same event logic is used for multiple controls.

3 Lambda Expression (Modern & Recommended)

Short and clean. Since EventHandler is a functional interface, you can use a lambda:

```
btn.setOnAction(e -> System.out.println("Clicked with lambda!"));
```

 Most preferred in modern JavaFX — clear and concise.

 Example: Basic Lambda Event

```
Button btnHello = new Button("Say Hello");
btnHello.setOnAction(e -> System.out.println("Hello there!"));
```

You can even interact with other UI elements inside:

```
Label status = new Label("Waiting...");
Button update = new Button("Update");

update.setOnAction(e -> status.setText("Updated at " + LocalTime.now()));
```

 addEventHandler() vs setOnAction() vs addEventFilter()

MethodPhase	Multiple Handlers?	Replaces Previous?	Common Use
setOnAction	Bubbling	No	Yes	Simple single-action controls (Button, MenuItem)
addEventHandler	Bubbling	Yes	No	Multiple handlers on same node
addEventFilter	Capturing	Yes	No	Intercept or block before reaching target

 Example: Event Propagation Demo

Here’s a simplified version of the big demo:

```
button.setOnAction(e -> System.out.println("Button setOnAction fired!"));

button.addEventHandler(ActionEvent.ACTION, e ->
    System.out.println("Button addEventHandler fired!"));

root.addEventFilter(MouseEvent.MOUSE_CLICKED, e ->
    System.out.println("Root capturing event..."));

root.addEventHandler(MouseEvent.MOUSE_CLICKED, e ->
    System.out.println("Root bubbling event..."));
```

Try these:

Click button: You'll see both handlers fire.

Ctrl + Click button: Consumed → doesn't reach root.

Double-click blue area: Captured → stops at filter.

⌨ Common Event Types

Event Type	Trigger Example
ActionEvent	Button click, Enter key in TextField
MouseEvent	Click, Move, Drag, Enter, Exit
KeyEvent	Key press, release, typed

ScrollEvent	Mouse wheel scroll
DragEvent	Drag-and-drop actions
WindowEvent	Stage close, show, hide

 Common Mistakes

 Forgetting to set the handler:

Writing the lambda but never calling setOnAction().

 Wrong event type:

Using MouseEvent for a button click instead of ActionEvent.

 Freezing the UI:

Running long tasks in handlers — always use threads or Platform.runLater().

 Overwriting handlers:

Calling setOnAction twice replaces the old one. Use addEventHandler() if you need multiple.

 Not consuming events:

Causes unwanted extra behavior higher up in the scene graph.

 Pro Tips

 Use lambda expressions for clean, readable event code.

 For complex logic, call a method:

button.setOnAction(this::handleClick);

 Use addEventFilter for input validation or blocking behavior.

 Use KeyEvent on Scene for global shortcuts like Ctrl+S.

 Use System.out.println to debug event flow.

 Practical Examples

1 Button Events (Click, Right-Click, Keyboard)

Left Click → Action event

Right Click → MouseEvent

Shift + Click → Combination event

Key Press → KeyEvent (like Enter or Space)

2 TextField Validation

Use focusedProperty() listener to validate when losing focus.

Use addEventFilter(KeyEvent.KEY_TYPED, ...) to block invalid characters (like letters in a numeric field).

 In Summary

Concept Phase MethodExample Use

Filter Capturing (Root → Target) addEventFilter()Block double-clicks, validate input

HandlerBubbling (Target → Root) addEventHandler() or setOnAction() Respond to clicks or keys

consume() — event.consume() Stop further propagation

Overview

This section teaches basic UI components (controls) you'll use in almost every JavaFX app — buttons, labels, input fields, etc.

Each control is:

Introduced (its purpose and properties)

Shown with a code demo

Combined with event handling to make it interactive.

 1. Button

 Purpose:

Used to trigger an action when clicked (like submitting a form, starting a process, etc.)

 Common Properties:

Property Description

text Button label

graphic Add an icon

disable Enables/disables button

defaultButton Reacts to ENTER key

cancelButton Reacts to ESC key

 Main Event:

ActionEvent — handled using setOnAction(e -> {...})

 Key Idea:

You can attach an image icon, change its text when clicked, or disable it temporarily while something runs.

 Example Summary:

A button labeled “Start Process” with a play icon.

When clicked:

Updates a label (“Process started!”)

Disables itself for 2 seconds (simulating work)

Re-enables after finishing.

Another button toggles its enable/disable state.

 Concepts learned: setOnAction, setDisable, Threads + Platform.runLater (to update UI safely from background).

 2. Label

 Purpose:

Displays text or an image — for info only (not editable).

 Common Properties:

Property	Description
text	The displayed text
graphic	Optional image/icon
contentDisplay	How text and image are arranged
wrapText	Wrap long text into multiple lines

 Example Summary:

A simple text label.

A long text label with wrapping and background color.

A label with an icon beside text.

A label with an icon above text.

 Concepts learned: setWrapText(true), setContentDisplay(ContentDisplay.LEFT/TOP), and using ImageView inside a label.

 3. TextField

 Purpose:

Single-line text input for user entry (e.g., name, email).

 Common Properties:

Property	Description
text	Current text
promptText	Placeholder text
editable	Allows editing
textProperty()	Observable property for listening changes

 Events:

ActionEvent → when ENTER is pressed

KeyEvent → when typing

textProperty().addListener() → real-time changes

 Example Summary:

A TextField with placeholder “Enter your full name here”.

Displays “Hello, [Name]!” dynamically as user types.

When Enter is pressed, logs the text.

 Concepts learned: live text listening, addListener, and text styling.

 4. PasswordField

 Purpose:

Like TextField but hides text (shows dots/asterisks).

 Events:

Same as TextField.

 Example Summary:

A simple login form (username + password).

“Sign In” button checks if username/password match “admin/secret”.

Shows success/failure message and clears the password.

 Concepts learned: secure input, event handling, grid layout.

 5. TextArea

 Purpose:

For multi-line text input (e.g., feedback form).

 Common Properties:

Property	Description
text	Content
wrapText	Wrap lines automatically
promptText	Placeholder
editable	Whether it’s editable

 Events:

Same as TextField — but pressing Enter adds a newline instead of triggering ActionEvent.

 Example Summary:

User writes feedback.

A live character counter updates as they type.

If >200 characters → turns red.

 Concepts learned: text listener, wrapText, and character counting.

 6. CheckBox

 Purpose:

Allows selecting or deselecting independent options.

 Common Properties:

Property	Description
text	Label
selected	True/False
allowIndeterminate	Optional third “unknown” state

 Events:

setOnAction

selectedProperty().addListener()

 Example Summary:

Three checkboxes: Email, SMS, and Dark Mode.

Updates a label showing which are ON/OFF.

Dark Mode can be “unknown” (indeterminate).

 Concepts learned: checkbox state handling, 3-state logic.

 7. RadioButton (with ToggleGroup)

 Purpose:

Choose one option from a group (mutually exclusive).

 Key Concept:

All buttons in the same ToggleGroup are linked — only one can be active.

 Events:

selectedToggleProperty().addListener() on the ToggleGroup.

 Example Summary:

Radio buttons for payment methods: Credit Card, PayPal, Bank Transfer.

Displays the currently selected option.

 Concepts learned: ToggleGroup, mutual exclusion, property listener.

 8. ComboBox

 Purpose:

Dropdown list (editable or not).

 Key Concept:

ObservableList holds all items.

 Events:

valueProperty().addListener() (recommended)

setOnAction() (when selected or Enter pressed)

 Example Summary:

Dropdown of countries.

Shows selected country.

Editable ComboBox allows user typing custom values.

 Concepts learned: dropdowns, editable inputs, custom values.

 9. ListView

 Purpose:

Displays a scrollable list (can select one or many items).

 Key Concept:

ObservableList holds data; SelectionModel manages which items are selected.

 Events:

selectedItemProperty().addListener() for single selection

selectedItemsProperty().addListener() for multiple

 Example Summary:

List of fruits.

Displays selected fruit.

Can switch between single or multiple selection mode.

 Concepts learned: working with list data and reacting to selections.

 Summary Table

Control	Input Type	Main Event	Special Feature
Button	Click	ActionEvent	Can be disabled or styled
Label	Display only	MouseEvent	Can show text + icon

TextField	Single-line input	ActionEvent / Listener	Real-time updates
PasswordField	Hidden input	ActionEvent	Secure entry
TextArea	Multi-line input	Listener	Word wrapping
CheckBox	True/False	ActionEvent	Multiple independent choices
RadioButton	One of many	ToggleGroup Listener	Mutually exclusive
ComboBox	Dropdown	valueProperty()	Editable option
ListView	List of items	selectedItemProperty()	Single or multiple selection

1. What Are Composite UI Controls?

These are **controls made up of multiple simple controls**, often used to display structured or interactive data — for example, a table showing rows and columns, or a slider for volume control.

□ 2. TableView

□ Purpose

Displays data in **rows and columns** — like an Excel table or a student list.

□ Key Parts

Concept	Meaning
Model Class (POJO)	Represents one row's data (like `Student`)
ObservableList	A list that automatically updates the UI when changed
TableColumn	Defines one column (e.g., "Name", "Age")
PropertyValueFactory	Connects a column to a model's getter method
Cell Factory	Customizes how cells look (e.g., colors, icons, buttons)

□ Example: Student Table

You have:

```
TableColumn<Student, String> idCol = new TableColumn<>("Student ID");
```

```
idCol.setCellValueFactory(new PropertyValueFactory<>("studentId"));
```

 This line links the “Student ID” column to the `getStudentId()` method in the `Student` class.

Custom rendering example:

```
enrolledCol.setCellFactory(column -> new TableCell<>() {
    @Override
    protected void updateItem(Boolean item, boolean empty) {
        if (empty || item == null) setText(null);
        else setText(item ? "Yes" : "No");
    }
});
```

 This shows **“Yes” in green** and **“No” in red** for each row.

Row Selection

```
studentTable.getSelectionModel().selectedItemProperty().addListener((obs,
oldVal, newVal) -> {
    if (newVal != null)
        System.out.println("Selected: " + newVal.getFirstName());
});
```

 Detects which row the user clicked.

---

3. TreeView

Purpose

Displays **hierarchical data** (parent-child relationships), like a file explorer.

Key Concept

- **TreeItem<T>** – each node (folder or file) is a TreeItem.
- **TreeView<T>** – the overall tree structure.

Example

```
TreeItem<String> rootItem = new TreeItem<>("MyProject");
TreeItem<String> srcFolder = new TreeItem<>("src");
srcFolder.getChildren().addAll(new TreeItem<>("Main.java"), new
TreeItem<>("Util.java"));
rootItem.getChildren().add(srcFolder);
```

```
TreeView<String> treeView = new TreeView<>(rootItem);
```

 Creates a tree like:

```
MyProject
 └── src
     ├── Main.java
     └── Util.java
```

---

4. ProgressBar

Purpose

Shows how much of a task is done (0 → 100%).

Example

```
ProgressBar bar = new ProgressBar(0);
Thread backgroundThread = new Thread(() -> {
    for (int i = 0; i <= 100; i++) {
        double progress = i / 100.0;
        Platform.runLater(() -> bar.setProgress(progress));
    }
});
```

 You must use `Platform.runLater()` because **only the main UI thread** can update UI elements.

---

5. Slider

Purpose

Lets users choose a value (e.g., volume).

Example

```
Slider slider = new Slider(0, 100, 50);
slider.valueProperty().addListener((obs, oldVal, newVal) -> {
    System.out.println("Volume: " + newVal.intValue());
});
```

 As the user drags, the value updates in real-time.

□ 6. DatePicker

□ Purpose

Lets users pick a date from a calendar.

□ Example

```
DatePicker datePicker = new DatePicker();
datePicker.setValue(LocalDate.now());
datePicker.valueProperty().addListener((obs, oldVal, newVal) -> {
    System.out.println("Selected: " + newVal);
});
```

□ 7. ColorPicker

□ Purpose

Lets users choose a color visually.

□ Example

```
ColorPicker picker = new ColorPicker(Color.LIGHTGRAY);
picker.setOnAction(e -> {
    Color c = picker.getValue();
    root.setStyle("-fx-background-color: #" + c.toString().substring(2, 8) +
";");
});
```

□ Changes the background color to what the user picks.

□ 8. Custom Modal Dialog (Confirm Box)

□ Purpose

Opens a small popup that blocks other interactions until closed — e.g., “Are you sure?”

□ How It Works

```
window.initModality(Modality.APPLICATION_MODAL);
window.showAndWait();
```

→ This pauses the main window until the popup closes.

□ Example

```
boolean result = ConfirmBox.display("Exit", "Are you sure?");
if (result) System.out.println("User clicked Yes");
```

□ Summary

Control	Purpose	Common Use
TableView	Tabular data	Student list, inventory
TreeView	Hierarchical data	File explorer
ProgressBar	Task progress	Loading files
Slider	Value control	Volume, brightness
DatePicker	Pick a date	Booking, scheduling
ColorPicker	Pick a color	Theme or design
Modal Dialog	Temporary popup	Confirmations